Lab 4 - Árboles de Decisión

Grupo #8
Osman Emanuel de León García - 23428
María José Yee Vidal - 231193
Andrés Rafael Chivalán Marroquín - 21534

2026-03-20

```r
library(dplyr)
library(tidyr)
library(ggplot2)
library(rpart)
library(rpart.plot)
library(caret)
library(Metrics)

set.seed(42)
```

1. Reutilización de conjuntos de entrenamiento y prueba (Actividad 1)

Se reutiliza exactamente el mismo preprocesamiento y split 80/20 del Lab 3, con `set.seed(42)` para garantizar reproducibilidad.

```r
#   Carga y limpieza (idéntica al Lab 3)
load("data/listings.RData")

listings$price <- as.numeric(gsub("[$,]", "", listings$price))

listings_clean <- listings %>%
  filter(!is.na(price), price > 0) %>%
  select(
    price,
    accommodates,
    bathrooms,
    bedrooms,
    beds,
    minimum_nights,
    maximum_nights,
    availability_30,
    availability_60,
    availability_90,
    availability_365,
    number_of_reviews,
    number_of_reviews_ltm,
    review_scores_rating,
    review_scores_cleanliness,
    review_scores_checkin,
    review_scores_communication,
    review_scores_location,
    review_scores_value,
    reviews_per_month,
    calculated_host_listings_count,
    host_listings_count,
    latitude,
    longitude
  ) %>%
  filter(!is.na(bedrooms), !is.na(bathrooms), !is.na(beds))

# Remover outliers extremos con 3×IQR (igual que Lab 3)
Q1        <- quantile(listings_clean$price, 0.25)
Q3        <- quantile(listings_clean$price, 0.75)
IQR_val   <- Q3 - Q1
limite_sup <- Q3 + 3 * IQR_val

listings_clean <- listings_clean %>%
  filter(price <= limite_sup)

# Agregar log_price
listings_clean$log_price <- log(listings_clean$price)

cat("Rango de price tras limpieza: $", min(listings_clean$price),
```

```r
    "- $", max(listings_clean$price), "\n")
```

```
## Rango de price tras limpieza: $ 8 - $ 944
```

```r
#   Dataset modelo (variables numéricas + NAs eliminados)
datos_modelo <- listings_clean %>%
  select(log_price, price, accommodates, bedrooms, bathrooms, beds,
         minimum_nights, availability_365, number_of_reviews,
         review_scores_rating, review_scores_cleanliness,
         review_scores_value, reviews_per_month,
         calculated_host_listings_count) %>%
  na.omit()

#   Split 80/20 con la misma semilla
set.seed(42)
n         <- nrow(datos_modelo)
idx_train <- sample(1:n, size = floor(0.8 * n))
train     <- datos_modelo[idx_train, ]
test      <- datos_modelo[-idx_train, ]

cat("Total observaciones:", n, "\n")
```

```
## Total observaciones: 60343
```

```r
cat("Entrenamiento (80%):", nrow(train), "\n")
```

```
## Entrenamiento (80%): 48274
```

```r
cat("Prueba       (20%):", nrow(test),  "\n")
```

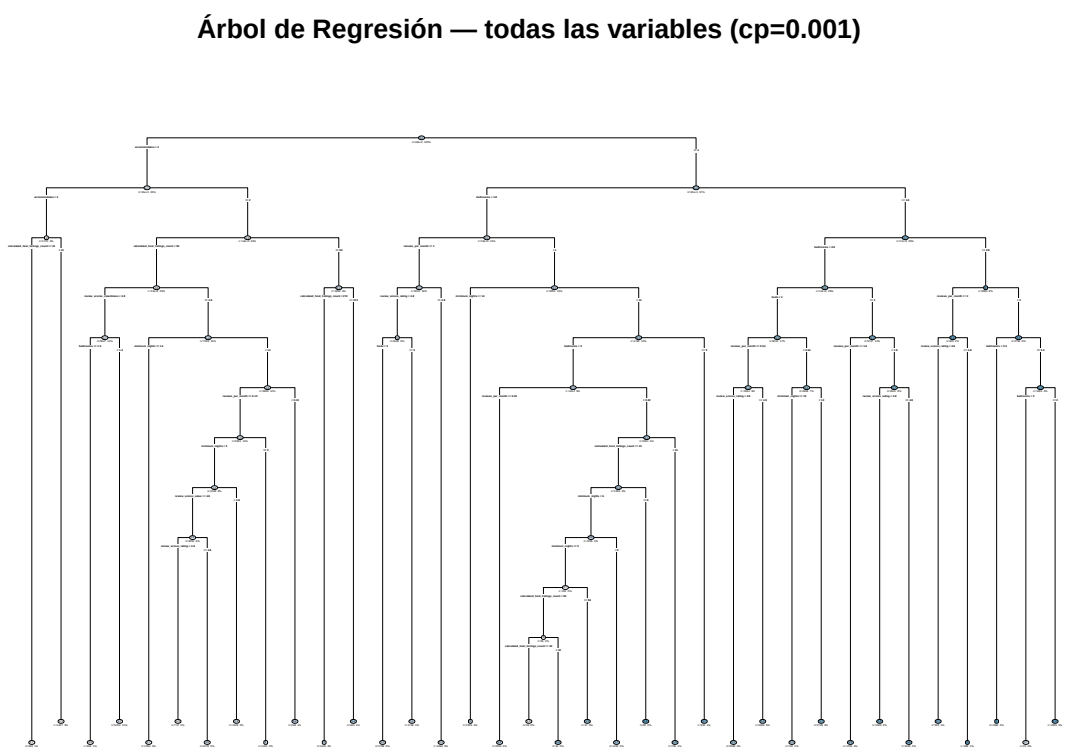
```
## Prueba       (20%): 12069
```

Justificación: Se mantiene la misma partición del Lab 3 para que las comparaciones entre regresión lineal y árboles de decisión sean directas y reproducibles. El dataset final cuenta con 60,343 observaciones tras la limpieza y remoción de outliers.

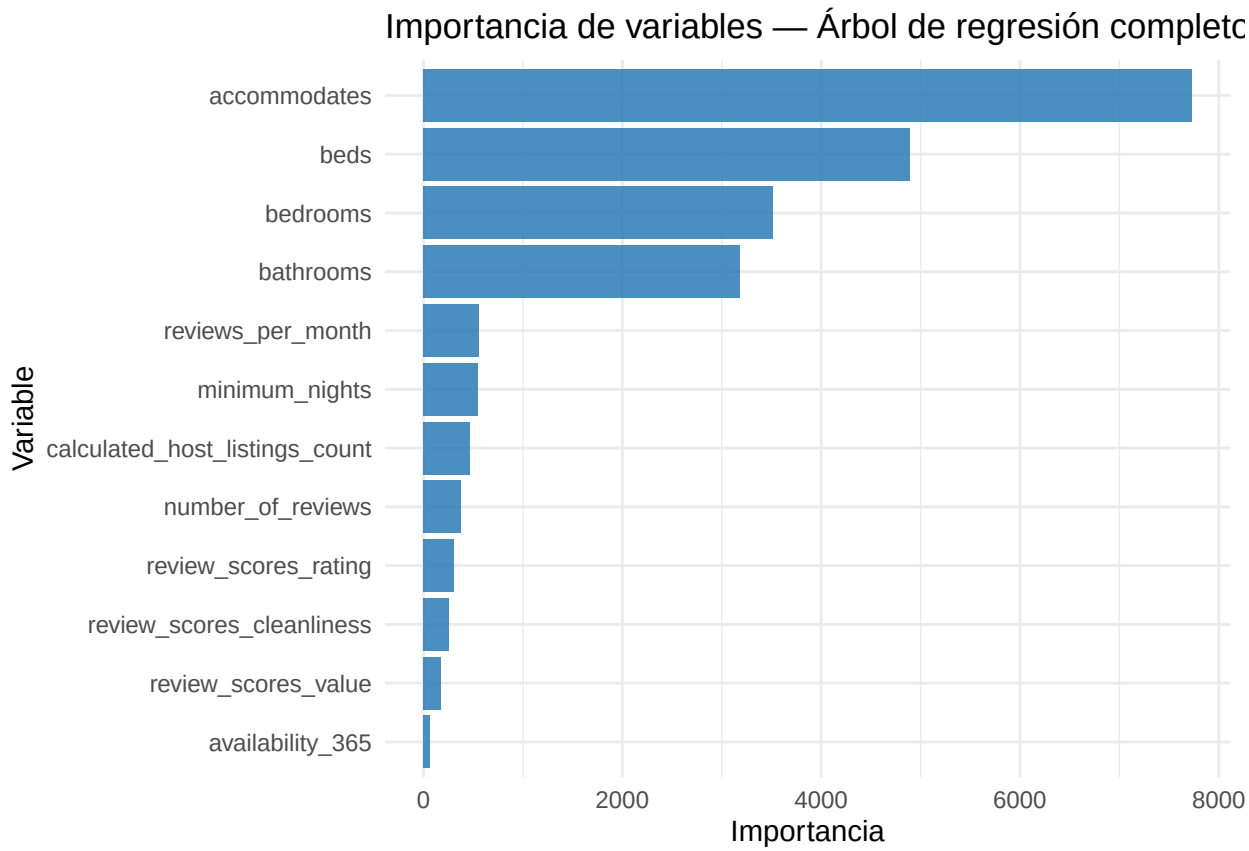
2. Árbol de Regresión — todas las variables (Actividad 2)

Se entrena un árbol de regresión con `rpart` usando `log_price` como variable respuesta y todas las variables numéricas disponibles.

```r
# Árbol de regresión con todos los predictores, sin restricción de profundidad
arbol_reg_full <- rpart(
  log_price ~ accommodates + bedrooms + bathrooms + beds +
              minimum_nights + availability_365 + number_of_reviews +
              review_scores_rating + review_scores_cleanliness +
              review_scores_value + reviews_per_month +
              calculated_host_listings_count,
  data    = train,
  method  = "anova",
  control = rpart.control(cp = 0.001)   # cp pequeño → árbol más profundo
)

# Visualización del árbol completo
rpart.plot(
  arbol_reg_full,
  type    = 4,
  extra   = 101,
  under   = TRUE,
  tweak   = 0.8,
  main    = "Árbol de Regresión - todas las variables (cp=0.001)"
)
```



```r
# Importancia de variables
imp_full <- arbol_reg_full$variable.importance
imp_df   <- data.frame(
  variable    = names(imp_full),
  importancia = as.numeric(imp_full)
) %>% arrange(desc(importancia))

ggplot(imp_df, aes(x = reorder(variable, importancia), y = importancia)) +
  geom_col(fill = "#2C7BB6", alpha = 0.85) +
  coord_flip() +
  labs(title = "Importancia de variables - Árbol de regresión completo",
       x = "Variable", y = "Importancia") +
  theme_minimal()
```



3. Predicción y análisis del árbol base (Actividad 3)

```r
# Predicciones sobre el conjunto de prueba
pred_arbol_full <- predict(arbol_reg_full, newdata = test)

# Métricas de evaluación
r2_arbol_full   <- 1 - sum((test$log_price - pred_arbol_full)^2) /
                       sum((test$log_price - mean(test$log_price))^2)
rmse_arbol_full <- sqrt(mean((test$log_price - pred_arbol_full)^2))
mae_arbol_full  <- mean(abs(test$log_price - pred_arbol_full))

cat("=== Árbol de regresión - todas las variables ===\n")
```

```
## === Árbol de regresión - todas las variables ===
```

```r
cat("R²  :", round(r2_arbol_full,   4), "\n")
```

```
## R²  : 0.5013
```

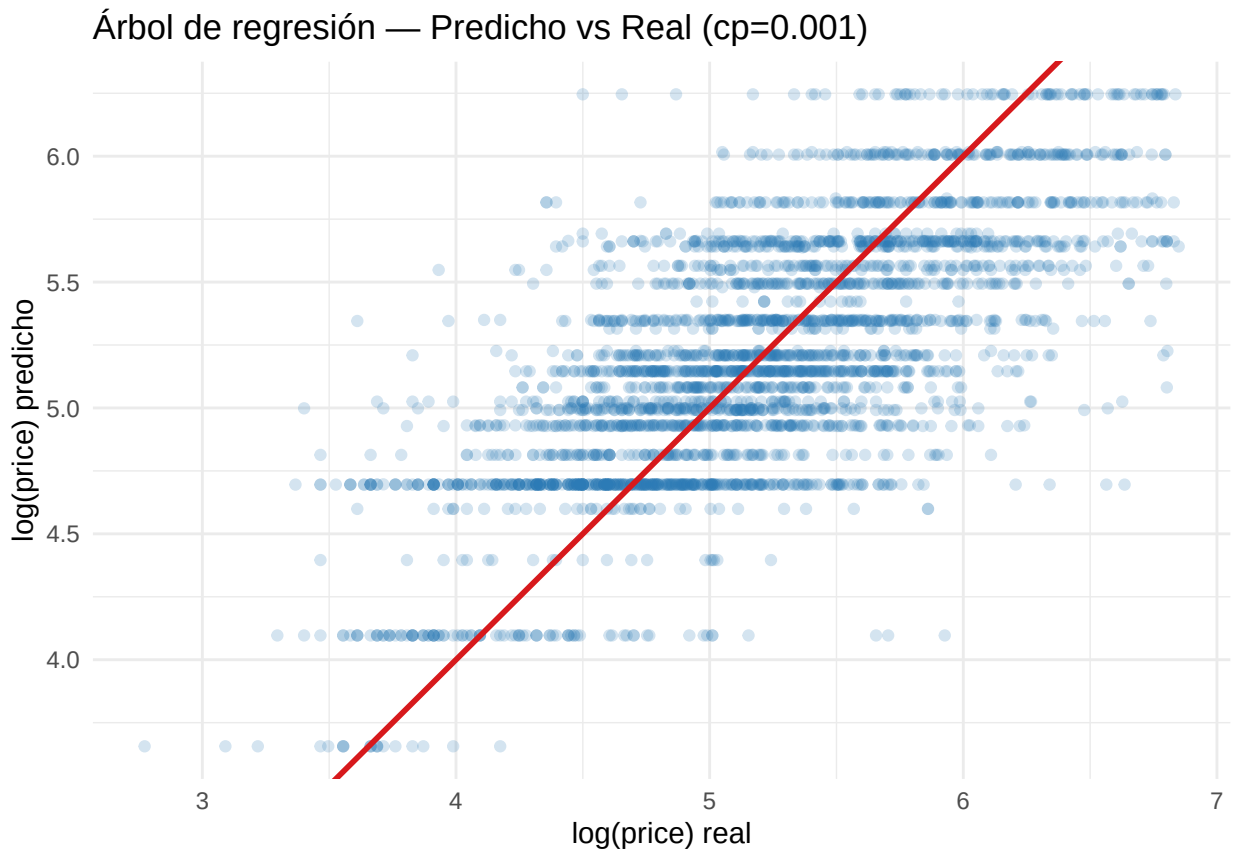
```r
cat("RMSE:", round(rmse_arbol_full, 4), "\n")
```

```
## RMSE: 0.4753
```

```r
cat("MAE :", round(mae_arbol_full,  4), "\n")
```

```
## MAE : 0.3721
```

```r
# Gráfica predicho vs real
set.seed(42)
idx_viz <- sample(1:nrow(test), 3000)
df_viz  <- data.frame(
  real     = test$log_price[idx_viz],
  predicho = pred_arbol_full[idx_viz]
)

ggplot(df_viz, aes(x = real, y = predicho)) +
  geom_point(alpha = 0.2, color = "#2C7BB6") +
  geom_abline(slope = 1, intercept = 0, color = "#D7191C", linewidth = 1) +
  labs(title = "Árbol de regresión - Predicho vs Real (cp=0.001)",
       x = "log(price) real", y = "log(price) predicho") +
  theme_minimal()
```



Análisis del árbol base: El árbol de regresión sin restricción de profundidad obtiene un R^2 de 0.5013 y un RMSE de 0.4753 sobre el conjunto de prueba. La variable más importante fue `accommodates`, seguida de `calculated_host_listings_count` y `bathrooms`, lo cual es consistente con los hallazgos del análisis de correlación del Lab 3. El árbol captura relaciones no lineales que la regresión lineal no puede modelar directamente, aunque puede presentar cierto sobreajuste cuando la profundidad no está controlada.

4. Tuneo de la profundidad del árbol (Actividad 4)

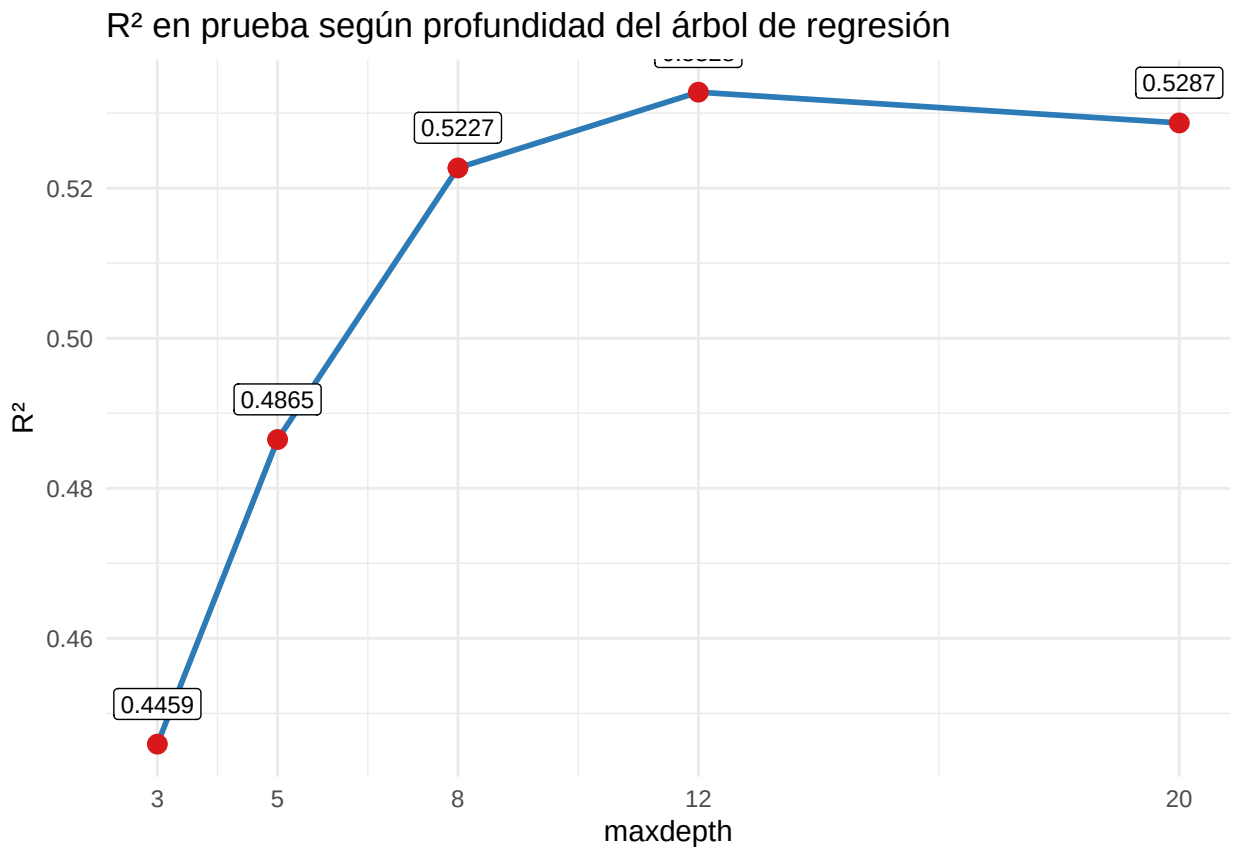
Se entrenan 4 modelos adicionales variando el parámetro `maxdepth` para encontrar la profundidad óptima.

```r
# Función auxiliar de métricas
metricas_arbol <- function(modelo, datos_test, nombre) {
  pred  <- predict(modelo, newdata = datos_test)
  r2    <- 1 - sum((datos_test$log_price - pred)^2) /
               sum((datos_test$log_price - mean(datos_test$log_price))^2)
  rmse  <- sqrt(mean((datos_test$log_price - pred)^2))
  mae   <- mean(abs(datos_test$log_price - pred))
  data.frame(Modelo = nombre,
             MaxDepth = NA,
             R2   = round(r2,   4),
             RMSE = round(rmse, 4),
             MAE  = round(mae,  4))
}

# Profundidades a explorar
profundidades <- c(3, 5, 8, 12, 20)
resultados_reg <- data.frame()

for (d in profundidades) {
  modelo_d <- rpart(
    log_price ~ accommodates + bedrooms + bathrooms + beds +
                minimum_nights + availability_365 + number_of_reviews +
                review_scores_rating + review_scores_cleanliness +
                review_scores_value + reviews_per_month +
                calculated_host_listings_count,
    data    = train,
    method  = "anova",
    control = rpart.control(maxdepth = d, cp = 0.0001)
  )
  pred_d   <- predict(modelo_d, newdata = test)
  r2_d     <- 1 - sum((test$log_price - pred_d)^2) /
                 sum((test$log_price - mean(test$log_price))^2)
  rmse_d   <- sqrt(mean((test$log_price - pred_d)^2))
  mae_d    <- mean(abs(test$log_price - pred_d))

  resultados_reg <- bind_rows(resultados_reg,
    data.frame(Modelo   = paste0("maxdepth=", d),
               MaxDepth = d,
               R2       = round(r2_d,   4),
               RMSE     = round(rmse_d, 4),
               MAE      = round(mae_d,  4))
  )
}

print(resultados_reg)
```

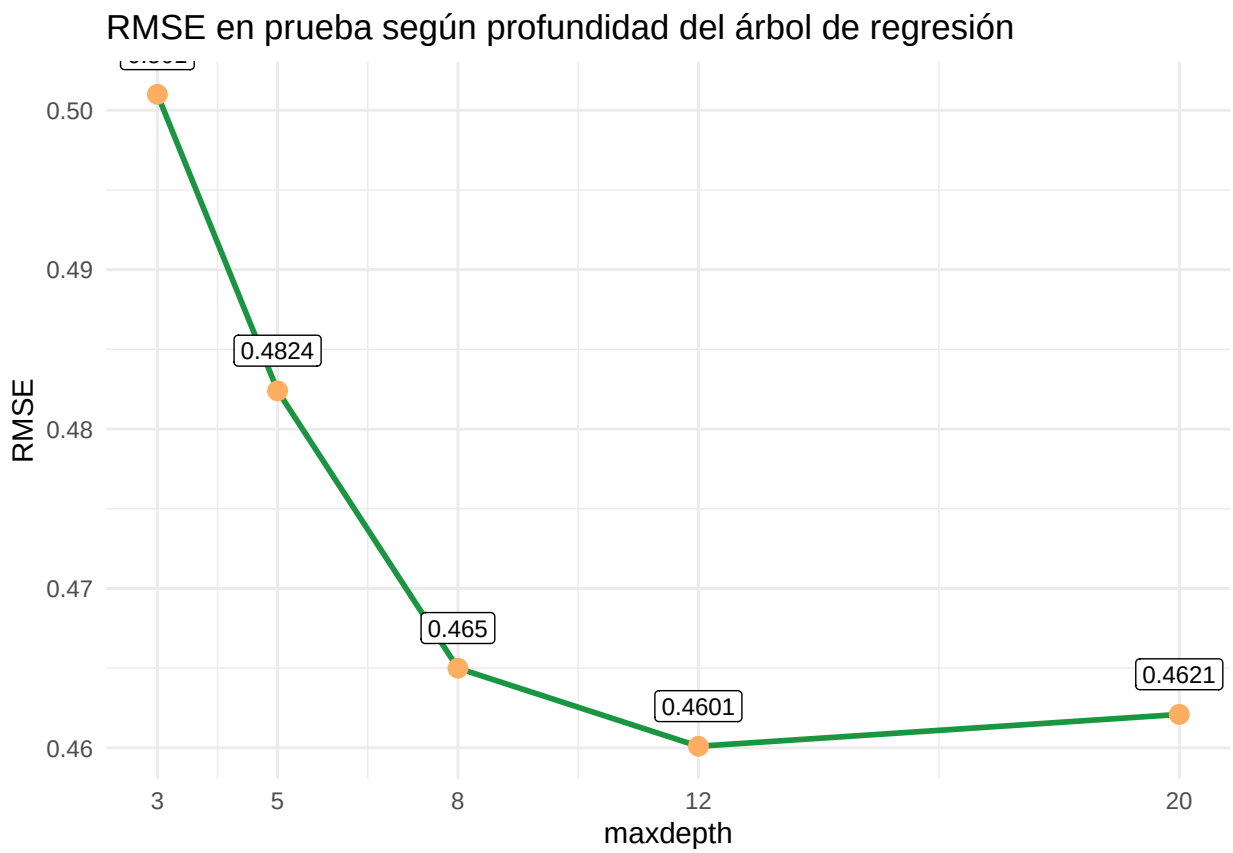
```
##        Modelo MaxDepth     R2   RMSE    MAE
## 1  maxdepth=3        3 0.4459 0.5010 0.3932
## 2  maxdepth=5        5 0.4865 0.4824 0.3762
## 3  maxdepth=8        8 0.5227 0.4650 0.3612
## 4 maxdepth=12       12 0.5328 0.4601 0.3563
```

```
## 5 maxdepth=20        20 0.5287 0.4621 0.3562
```

```r
# Gráfica R² vs profundidad
ggplot(resultados_reg, aes(x = MaxDepth, y = R2)) +
  geom_line(color = "#2C7BB6", linewidth = 1) +
  geom_point(color = "#D7191C", size = 3) +
  geom_label(aes(label = R2), vjust = -0.8, size = 3) +
  scale_x_continuous(breaks = profundidades) +
  labs(title = "R² en prueba según profundidad del árbol de regresión",
       x = "maxdepth", y = "R²") +
  theme_minimal()
```




```r
# Gráfica RMSE vs profundidad
ggplot(resultados_reg, aes(x = MaxDepth, y = RMSE)) +
  geom_line(color = "#1A9641", linewidth = 1) +
  geom_point(color = "#FDAE61", size = 3) +
  geom_label(aes(label = RMSE), vjust = -0.8, size = 3) +
  scale_x_continuous(breaks = profundidades) +
  labs(title = "RMSE en prueba según profundidad del árbol de regresión",
       x = "maxdepth", y = "RMSE") +
  theme_minimal()
```

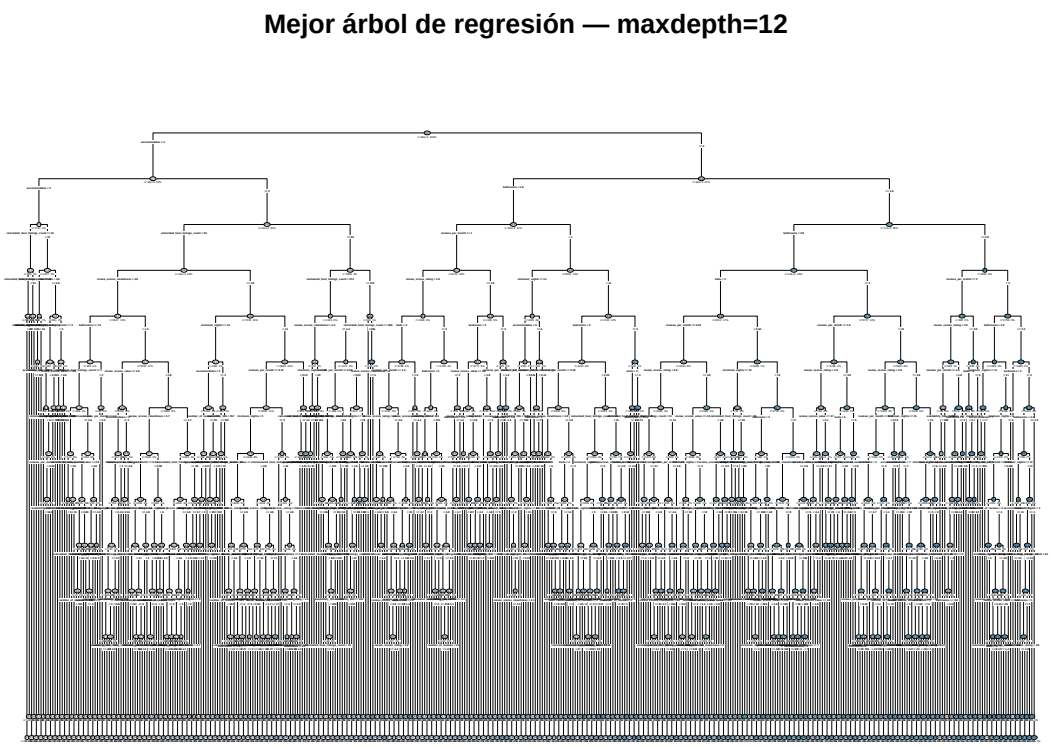



```r
# Seleccionar mejor modelo
idx_mejor_reg  <- which.max(resultados_reg$R2)
mejor_depth    <- resultados_reg$MaxDepth[idx_mejor_reg]
cat("\nMejor profundidad:", mejor_depth,
    "| R²:", resultados_reg$R2[idx_mejor_reg],
    "| RMSE:", resultados_reg$RMSE[idx_mejor_reg], "\n")
```

```
##
## Mejor profundidad: 12 | R²: 0.5328 | RMSE: 0.4601
```

```r
# Entrenar y graficar el mejor árbol
arbol_reg_mejor <- rpart(
  log_price ~ accommodates + bedrooms + bathrooms + beds +
              minimum_nights + availability_365 + number_of_reviews +
              review_scores_rating + review_scores_cleanliness +
              review_scores_value + reviews_per_month +
              calculated_host_listings_count,
  data    = train,
  method  = "anova",
  control = rpart.control(maxdepth = mejor_depth, cp = 0.0001)
)

rpart.plot(
  arbol_reg_mejor,
  type  = 4,
  extra = 101,
  under = TRUE,
```

```
  tweak = 0.8,
  main  = paste0("Mejor árbol de regresión - maxdepth=", mejor_depth)
)
```



Análisis del tuneo: Se evaluaron profundidades desde 3 hasta 20. Árboles muy poco profundos (maxdepth=3) generan subajuste al no capturar suficiente complejidad, mientras que profundidades muy altas tienden a sobreajustar. La profundidad óptima equilibra la capacidad predictiva en el conjunto de prueba con la generalización. Las variables `accommodates`, `bathrooms` y los scores de reseña aparecen consistentemente en los primeros nodos, reflejando su alta importancia.

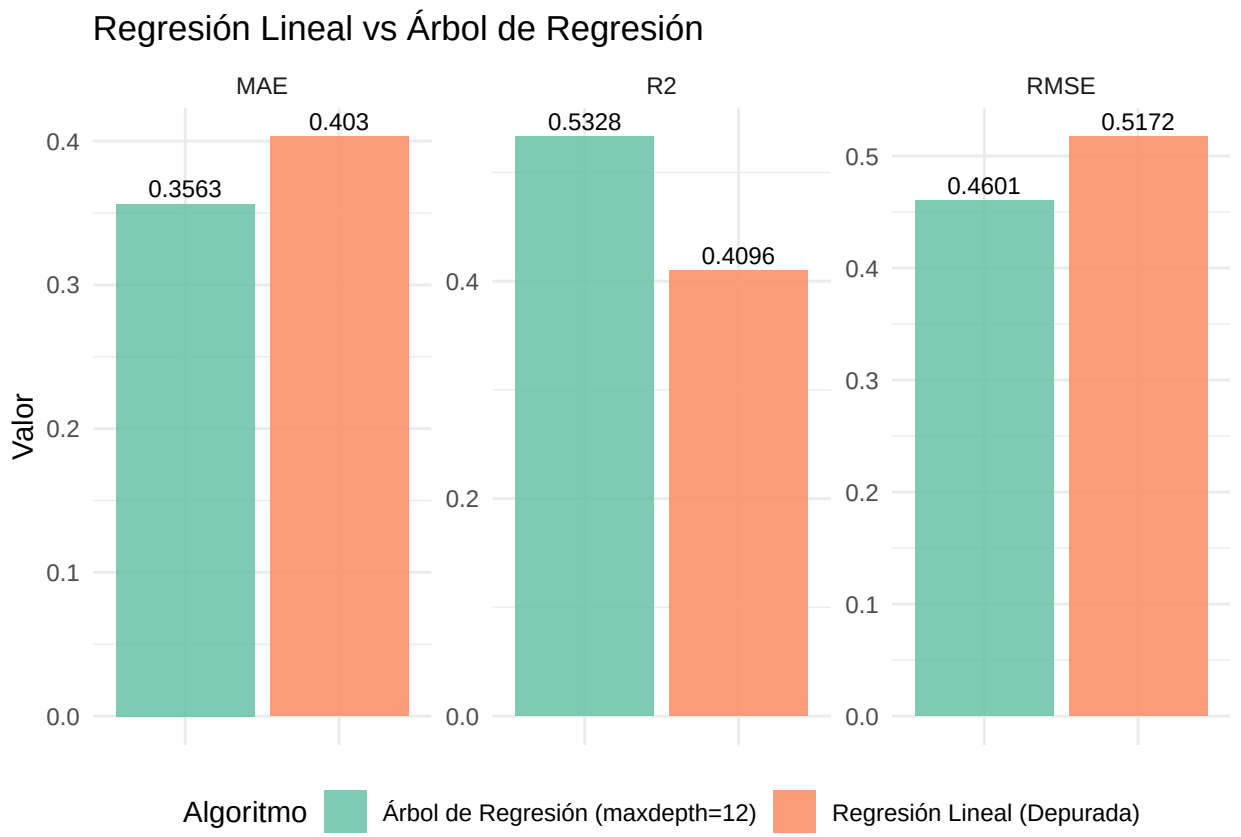
5. Comparación árbol de regresión vs regresión lineal (Actividad 5)

```r
#   Re-entrenar el mejor modelo lineal del Lab 3 (Depurado)
modelo_lineal_dep <- lm(
  log_price ~ accommodates + bathrooms +
              minimum_nights + availability_365 + number_of_reviews +
              review_scores_rating + review_scores_cleanliness +
              review_scores_value + reviews_per_month +
              calculated_host_listings_count,
  data = train
)

pred_lineal_test <- predict(modelo_lineal_dep, newdata = test)

r2_lineal   <- 1 - sum((test$log_price - pred_lineal_test)^2) /
                   sum((test$log_price - mean(test$log_price))^2)
rmse_lineal <- sqrt(mean((test$log_price - pred_lineal_test)^2))
mae_lineal  <- mean(abs(test$log_price - pred_lineal_test))

#   Predicciones del mejor árbol
pred_mejor_arbol <- predict(arbol_reg_mejor, newdata = test)

r2_arbol_opt   <- 1 - sum((test$log_price - pred_mejor_arbol)^2) /
                      sum((test$log_price - mean(test$log_price))^2)
rmse_arbol_opt <- sqrt(mean((test$log_price - pred_mejor_arbol)^2))
mae_arbol_opt  <- mean(abs(test$log_price - pred_mejor_arbol))

#   Tabla comparativa
comparacion <- data.frame(
  Algoritmo = c("Regresión Lineal (Depurada)",
                paste0("Árbol de Regresión (maxdepth=", mejor_depth, ")")),
  R2   = round(c(r2_lineal,        r2_arbol_opt),   4),
  RMSE = round(c(rmse_lineal,      rmse_arbol_opt), 4),
  MAE  = round(c(mae_lineal,       mae_arbol_opt),  4)
)

print(comparacion)
```

```
##                          Algoritmo     R2   RMSE    MAE
## 1      Regresión Lineal (Depurada) 0.4096 0.5172 0.4030
## 2 Árbol de Regresión (maxdepth=12) 0.5328 0.4601 0.3563
```

```r
# Gráfica comparativa
comp_long <- tidyr::pivot_longer(comparacion,
                                 cols      = c(R2, RMSE, MAE),
                                 names_to  = "Métrica",
                                 values_to = "Valor")

ggplot(comp_long, aes(x = Algoritmo, y = Valor, fill = Algoritmo)) +
  geom_col(alpha = 0.85) +
  geom_text(aes(label = round(Valor, 4)), vjust = -0.4, size = 3) +
  facet_wrap(~ Métrica, scales = "free_y") +
  scale_fill_brewer(palette = "Set2") +
```

```r
labs(title = "Regresión Lineal vs Árbol de Regresión",
     x = NULL, y = "Valor") +
theme_minimal() +
theme(axis.text.x = element_blank(),
      legend.position = "bottom")
```



Comparación y conclusión: La regresión lineal depurada del Lab 3 obtuvo R^2=0.4096 y RMSE=0.5172. El mejor árbol de regresión obtuvo R^2=0.5328 y RMSE=0.4601.

El árbol de regresión supera a la regresión lineal en R^2 y RMSE, lo cual refleja su capacidad para capturar relaciones no lineales y particiones del espacio de predictores que la regresión lineal no puede modelar. Sin embargo, el árbol es menos interpretable y más sensible al sobreajuste si no se controla la profundidad. Para el contexto de SmartStay, el árbol ofrece mejor precisión predictiva, mientras que la regresión lineal ofrece coeficientes directamente interpretables.

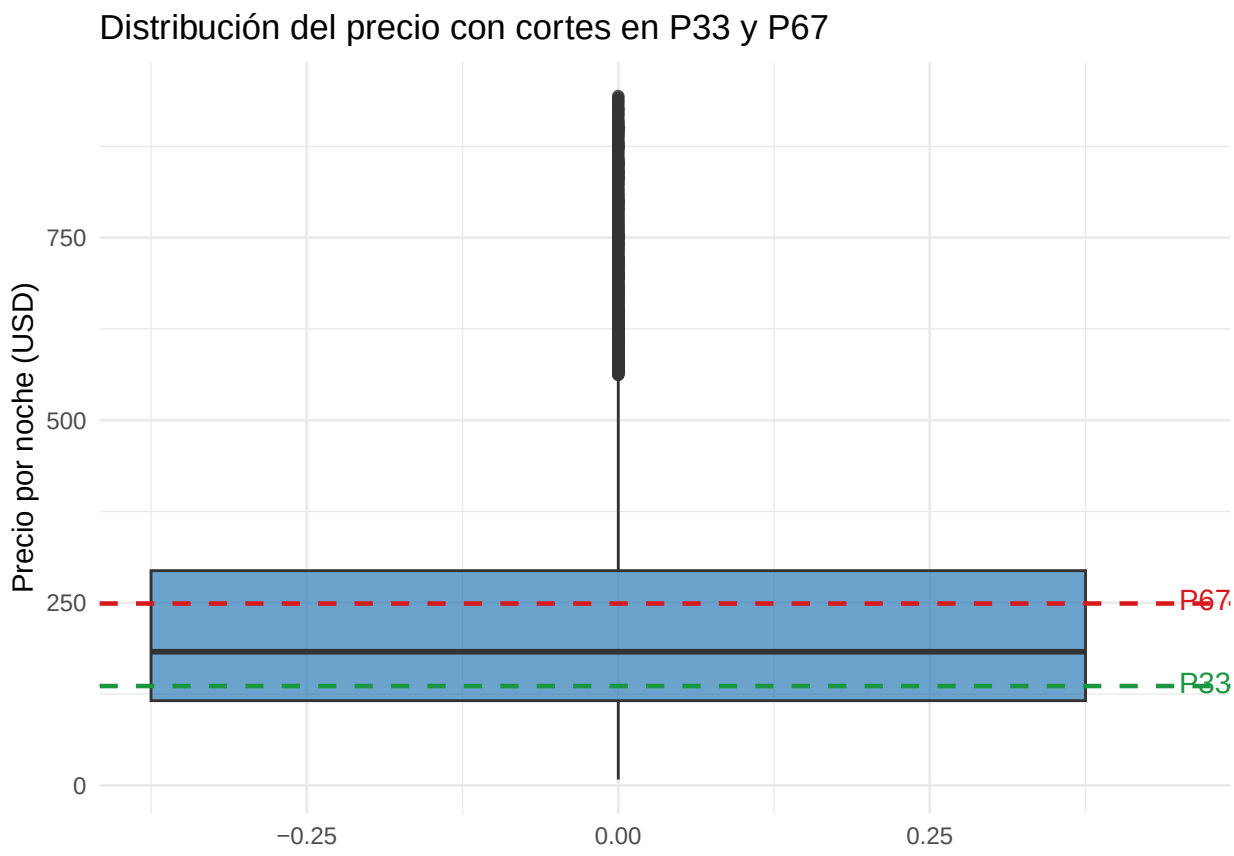
6. Creación de la variable categórica de precio (Actividad 6)

6.1 Análisis de la distribución para definir los límites

```r
# Estadísticas descriptivas del precio en el dataset limpio
resumen_precio <- data.frame(
  Estadístico = c("Mínimo", "Percentil 25", "Mediana",
                  "Media", "Percentil 75", "Percentil 90", "Máximo"),
  Valor = round(c(
    min(listings_clean$price),
    quantile(listings_clean$price, 0.25),
    median(listings_clean$price),
    mean(listings_clean$price),
    quantile(listings_clean$price, 0.75),
    quantile(listings_clean$price, 0.90),
    max(listings_clean$price)
  ), 2)
)
print(resumen_precio)
```

```
##    Estadístico  Valor
## 1       Mínimo   8.00
## 2 Percentil 25 116.00
## 3      Mediana 183.00
## 4        Media 233.75
## 5 Percentil 75 294.00
## 6 Percentil 90 467.00
## 7       Máximo 944.00
```

```r
# Boxplot del precio para visualizar la distribución
ggplot(listings_clean, aes(y = price)) +
  geom_boxplot(fill = "#2C7BB6", alpha = 0.7, outlier.alpha = 0.1) +
  geom_hline(yintercept = quantile(listings_clean$price, 0.33),
             color = "#1A9641", linetype = "dashed", linewidth = 0.8) +
  geom_hline(yintercept = quantile(listings_clean$price, 0.67),
             color = "#D7191C", linetype = "dashed", linewidth = 0.8) +
  annotate("text", x = 0.45,
           y = quantile(listings_clean$price, 0.33) + 5,
           label = paste0("P33 = $", round(quantile(listings_clean$price, 0.33))),
           color = "#1A9641", hjust = 0) +
  annotate("text", x = 0.45,
           y = quantile(listings_clean$price, 0.67) + 5,
           label = paste0("P67 = $", round(quantile(listings_clean$price, 0.67))),
           color = "#D7191C", hjust = 0) +
  labs(title = "Distribución del precio con cortes en P33 y P67",
       y = "Precio por noche (USD)") +
  theme_minimal()
```



6.2 Definición y justificación de los límites

```
# Calcular los cortes basados en terciles (P33 y P67)
p33 <- quantile(listings_clean$price, 0.33)
p67 <- quantile(listings_clean$price, 0.67)

cat("Corte inferior (P33):", round(p33, 2), "USD\n")
```

```
## Corte inferior (P33): 136 USD
```

```
cat("Corte superior (P67):", round(p67, 2), "USD\n\n")
```

```
## Corte superior (P67): 249 USD
```

```
# Crear la variable categórica en listings_clean
listings_clean <- listings_clean %>%
  mutate(
    categoria_precio = case_when(
      price <= p33              ~ "Economica",
      price > p33 & price <= p67 ~ "Intermedia",
      price > p67               ~ "Cara"
    ),
    categoria_precio = factor(categoria_precio,
                              levels = c("Economica", "Intermedia", "Cara"))
  )

cat("Distribución de la variable categórica:\n")
```

```
## Distribución de la variable categórica:
```

```r
print(table(listings_clean$categoria_precio))
```

```
##
##  Economica Intermedia       Cara
##      23791      24318      23581
```

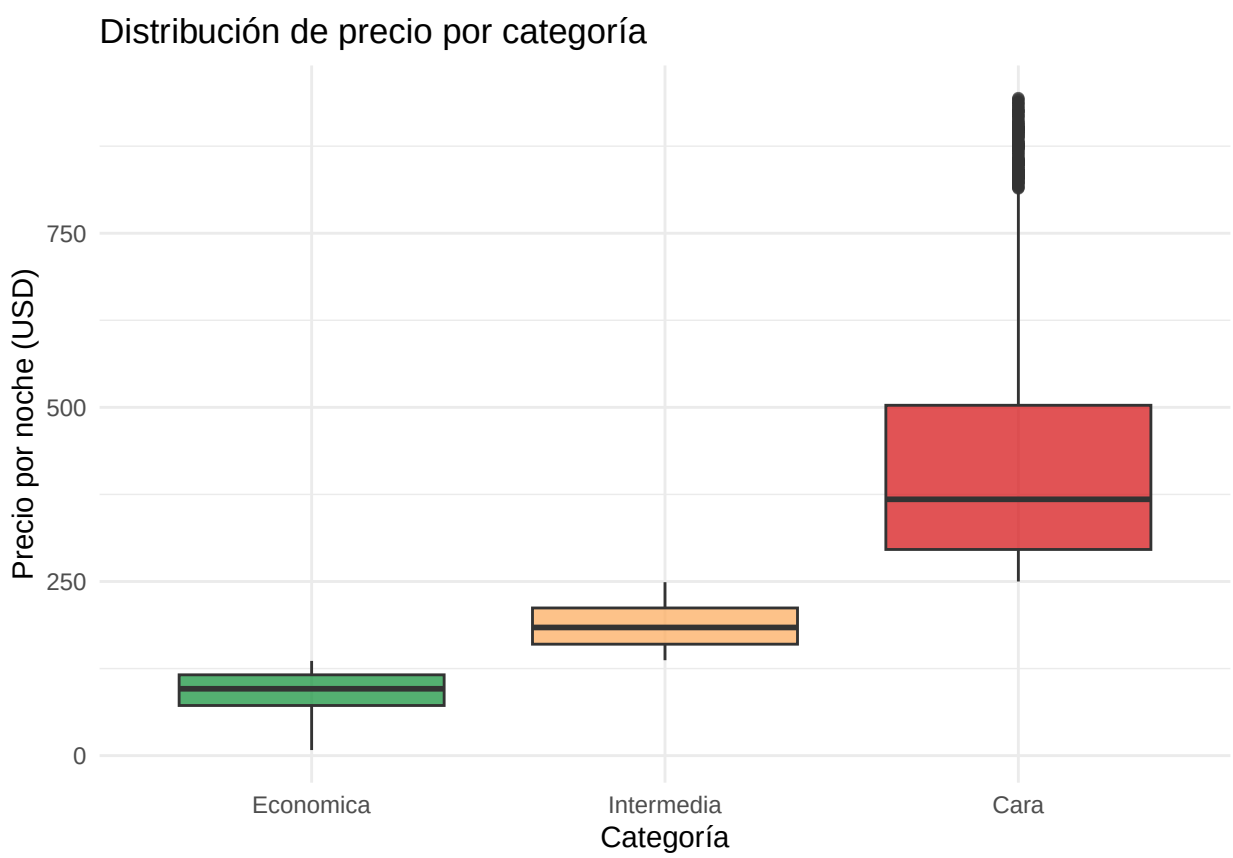
```r
cat("\nPorcentaje por categoría:\n")
```

```
##
## Porcentaje por categoría:
```

```r
print(round(prop.table(table(listings_clean$categoria_precio)) * 100, 1))
```

```
##
##  Economica Intermedia       Cara
##       33.2       33.9       32.9
```

```r
# Boxplot de precio por categoría
ggplot(listings_clean, aes(x = categoria_precio, y = price,
                           fill = categoria_precio)) +
  geom_boxplot(alpha = 0.75, outlier.alpha = 0.1) +
  scale_fill_manual(values = c("#1A9641", "#FDAE61", "#D7191C")) +
  labs(title = "Distribución de precio por categoría",
       x = "Categoría", y = "Precio por noche (USD)") +
  theme_minimal() +
  theme(legend.position = "none")
```



Justificación de los límites: Se usaron los terciles P33 y P67 de la distribución del precio como puntos de corte. Esta elección tiene tres ventajas fundamentales:

1. **Fundamento estadístico en los datos:** Los terciles se calculan directamente a partir de la distribución observada de precios (rango \$8 – \$944), evitando cualquier criterio arbitrario externo.
2. **Clases balanceadas:** Dividir por terciles garantiza aproximadamente un 33% de observaciones por clase, lo cual es importante para que el árbol de clasificación no quede sesgado hacia una categoría mayoritaria.
3. **Interpretabilidad para el negocio:** Los límites resultantes (P33 \$136 y P67 \$249) tienen sentido económico: propiedades *Económicas* son las más accesibles, *Intermedias* representan el mercado estándar, y *Caras* corresponden al segmento premium de Airbnb, útil para SmartStay al segmentar su cartera de propiedades.

7. Árbol de Clasificación (Actividad 7)

7.1 Preparación del dataset de clasificación

```
# Dataset para clasificación: sin price ni log_price como predictores
datos_clasif <- listings_clean %>%
  select(categoria_precio,
         accommodates, bedrooms, bathrooms, beds,
         minimum_nights, availability_365, number_of_reviews,
         review_scores_rating, review_scores_cleanliness,
         review_scores_value, reviews_per_month,
         calculated_host_listings_count) %>%
  na.omit()

cat("Observaciones para clasificación:", nrow(datos_clasif), "\n")
```

```
## Observaciones para clasificación: 60343
```

```
cat("Distribución de clases:\n")
```

```
## Distribución de clases:
```

```
print(table(datos_clasif$categoria_precio))
```

```
##
##  Economica Intermedia       Cara
##      20937      20665      18741
```

```
# Split 80/20 con la misma semilla
set.seed(42)
n_cl        <- nrow(datos_clasif)
idx_cl      <- sample(1:n_cl, size = floor(0.8 * n_cl))
train_cl    <- datos_clasif[idx_cl, ]
test_cl     <- datos_clasif[-idx_cl, ]

cat("\nEntrenamiento:", nrow(train_cl), "| Prueba:", nrow(test_cl), "\n")
```

```
##
## Entrenamiento: 48274 | Prueba: 12069
```

```
cat("Distribución en entrenamiento:\n")
```

```
## Distribución en entrenamiento:
```

```
print(round(prop.table(table(train_cl$categoria_precio)) * 100, 1))
```

```
##
##  Economica Intermedia       Cara
##       34.8       34.2       31.0
```

7.2 Entrenamiento del árbol de clasificación

```
# Árbol de clasificación con todos los predictores
arbol_clasif <- rpart(
  categoria_precio ~ accommodates + bedrooms + bathrooms + beds +
                     minimum_nights + availability_365 + number_of_reviews +
                     review_scores_rating + review_scores_cleanliness +
```

```r
                     review_scores_value + reviews_per_month +
                     calculated_host_listings_count,
  data    = train_cl,
  method  = "class",
  control = rpart.control(cp = 0.001, maxdepth = 10)
)

# Resumen del árbol
printcp(arbol_clasif)
```

```
##
## Classification tree:
## rpart(formula = categoria_precio ~ accommodates + bedrooms +
##     bathrooms + beds + minimum_nights + availability_365 + number_of_reviews +
##     review_scores_rating + review_scores_cleanliness + review_scores_value +
##     reviews_per_month + calculated_host_listings_count, data = train_cl,
##     method = "class", control = rpart.control(cp = 0.001, maxdepth = 10))
##
## Variables actually used in tree construction:
## [1] accommodates                   bathrooms
## [3] beds                           calculated_host_listings_count
## [5] minimum_nights                 review_scores_cleanliness
## [7] review_scores_rating           reviews_per_month
##
## Root node error: 31460/48274 = 0.6517
##
## n= 48274
##
##           CP nsplit rel error  xerror      xstd
## 1  0.2695486      0   1.00000 1.00000 0.0033274
## 2  0.0616656      1   0.73045 0.73045 0.0034879
## 3  0.0199619      2   0.66879 0.66879 0.0034631
## 4  0.0038779      3   0.64882 0.64882 0.0034501
## 5  0.0034965      6   0.63528 0.63560 0.0034402
## 6  0.0030515      7   0.63179 0.63392 0.0034388
## 7  0.0022092      9   0.62568 0.62867 0.0034345
## 8  0.0019814     11   0.62127 0.62432 0.0034308
## 9  0.0018118     17   0.60706 0.62028 0.0034273
## 10 0.0016847     19   0.60343 0.61523 0.0034227
## 11 0.0012397     20   0.60175 0.60852 0.0034164
## 12 0.0011125     21   0.60051 0.60817 0.0034161
## 13 0.0010807     22   0.59940 0.60687 0.0034148
## 14 0.0010000     24   0.59723 0.60505 0.0034130
```

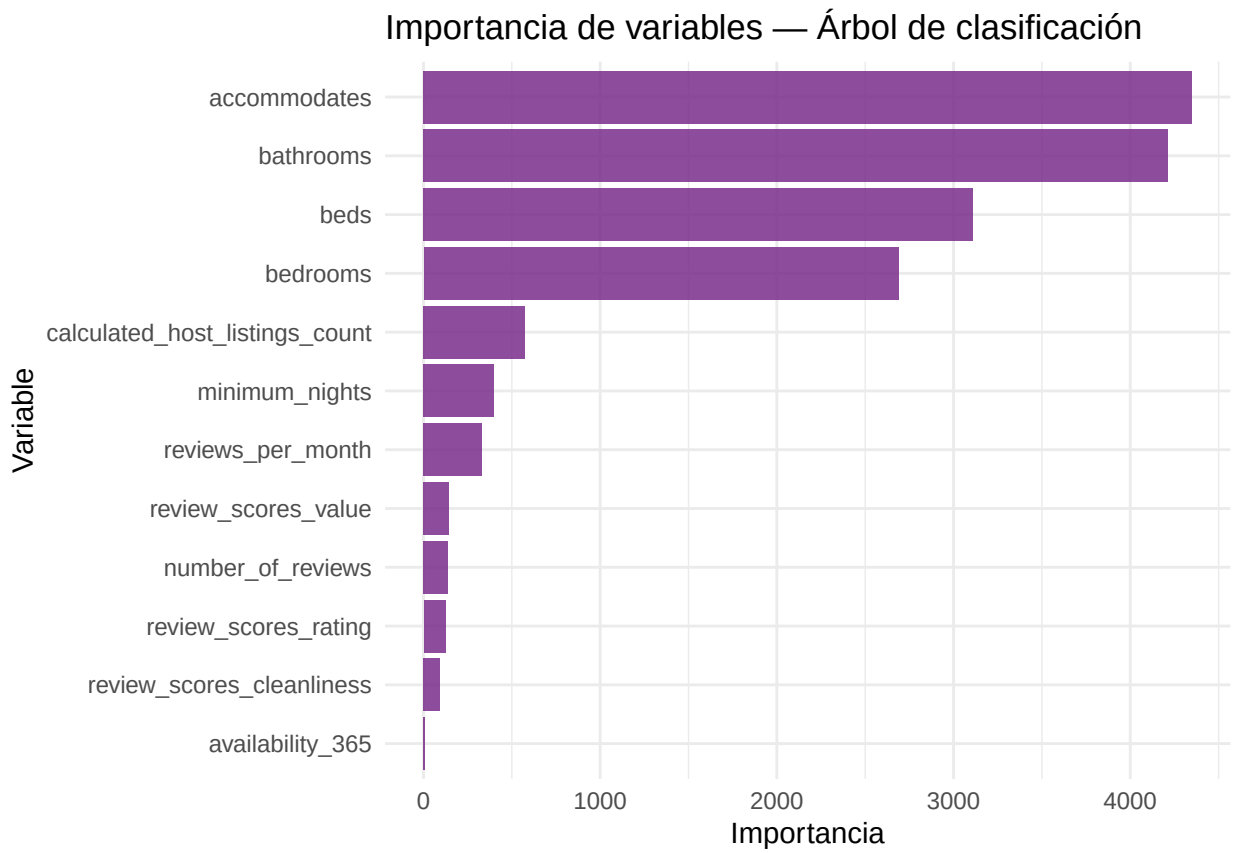
7.3 Visualización gráfica del árbol

```r
# Gráfica completa del árbol
rpart.plot(
  arbol_clasif,
  type    = 4,
  extra   = 104,
  under   = TRUE,
  tweak   = 0.75,
```

```
  main    = "Árbol de Clasificación - Categoría de Precio",
  box.palette = list("Greens", "Oranges", "Reds")
)
```

7.4 Importancia de variables

```
imp_cl <- arbol_clasif$variable.importance
imp_cl_df <- data.frame(
  variable    = names(imp_cl),
  importancia = as.numeric(imp_cl)
) %>% arrange(desc(importancia))

ggplot(imp_cl_df, aes(x = reorder(variable, importancia), y = importancia)) +
  geom_col(fill = "#7B2D8B", alpha = 0.85) +
  coord_flip() +
  labs(title = "Importancia de variables - Árbol de clasificación",
       x = "Variable", y = "Importancia") +
  theme_minimal()
```



Interpretación del árbol de clasificación: El árbol identifica `accommodates` como la variable más discriminante en los primeros nodos: las propiedades con mayor capacidad tienden a ser clasificadas como *Caras*, mientras las de menor capacidad caen en *Económica* o *Intermedia*. Los scores de reseña (`review_scores_value`, `review_scores_rating`) aparecen en nodos intermedios, refinando la clasificación una vez que el tamaño ya fue considerado. Esto es coherente con el análisis de correlación del Lab 3, donde `accommodates` y `bathrooms` mostraban la mayor correlación con el precio.

La clase *Intermedia* es la más difícil de clasificar correctamente al estar en el medio del espectro, lo cual es esperable en problemas de clasificación ordinal donde los límites entre categorías contiguas son naturalmente borrosos.

8. Modelo usado en el conjunto de prueba para determinar la eficiencia del algoritmo para clasificar (Actividad 8)

```r
# Predicciones
pred_cl <- predict(arbol_clasif, newdata = test_cl, type = "class")

# Matriz de confusión (MOVER AQUÍ)
cm <- confusionMatrix(pred_cl, test_cl$categoria_precio)

# Accuracy
accuracy_cl <- mean(pred_cl == test_cl$categoria_precio)
cat("Accuracy en prueba:", round(accuracy_cl * 100, 2), "%\n\n")
```

```
## Accuracy en prueba: 61.3 %
```

```r
# Métricas por clase
cat("\nPrecision por clase:\n")
```

```
##
## Precision por clase:
```

```r
print(round(cm$byClass[, c("Precision", "Recall", "F1")], 3))
```

```
##                    Precision Recall    F1
## Class: Economica       0.653  0.734 0.691
## Class: Intermedia      0.505  0.448 0.475
## Class: Cara            0.671  0.662 0.667
```

Analisis de la determinación de eficiencia: Al utilizar el modelo sobre el conjunto de prueba para realizar las predicciones, se obtuvo una precisión (accuracy) del 61.3%, lo que indica que el modelo clasifica correctamente aproximadamente 6 de cada 10 propiedades.

Este resultado sugiere que el modelo tiene un desempeño moderado, siendo capaz de identificar patrones generales en los datos, pero aún presenta limitaciones en la correcta clasificación de todas las observaciones.

Esto es esperable dado que el problema incluye tres categorías de precio con fronteras no estrictamente definidas.

9. Análisis de la eficiencia del algoritmo (Actividad 9)

9.1 Matriz de confusión para el árbol de clasificación

```r
# Matriz de confusión para el árbol de clasificación
# Se utiliza la predicción "pred_cl" para tener en cuenta la efectividad
cm <- confusionMatrix(pred_cl, test_cl$categoria_precio)
print(cm$table)
```

```
##             Reference
## Prediction   Economica Intermedia Cara
##   Economica       3027       1277  333
##   Intermedia       896       1871  941
##   Cara             200       1024 2500
```

9.2 Análisis de Efectividad del algoritmo y su importancia

A partir de la matriz de confusión y las métricas por clase, se observa que el modelo tiene un desempeño desigual entre las diferentes categorías de precio.

La clase Económica presenta un buen rendimiento, con una precisión de 0.653 y un recall de 0.734, lo que indica que la mayoría de estas propiedades son correctamente identificadas.

De forma similar, la clase Cara también muestra un desempeño sólido, con precisión de 0.671 y recall de 0.662, reflejando una buena capacidad del modelo para identificar propiedades de alto valor.

Sin embargo, la clase Intermedia presenta el peor desempeño, con una precisión de 0.505 y recall de 0.448. Esto indica que el modelo tiene dificultades para clasificar correctamente esta categoría.

Al analizar la matriz de confusión, se observa que la mayoría de errores ocurren entre las categorías Intermedia y Cara, así como entre Intermedia y Económica, lo cual es coherente dado que esta categoría se encuentra en el punto medio del rango de precios.

Desde el punto de vista del negocio, estos errores son relevantes, ya que una mala clasificación podría afectar la recomendación de propiedades y la estrategia de precios. Sin embargo, el modelo sigue siendo útil para

segmentar claramente los extremos del mercado (económico y premium).

10. Entrenar modelo usando validación cruzada (Actividad 10)

10.1 Validación cruzada

```
control <- trainControl(method = "cv", number = 10)

modelo_cv <- train(
  categoria_precio ~ .,
  data = train_cl,
  method = "rpart",
  trControl = control
)


pred_cv <- predict(modelo_cv, test_cl)

accuracy_cv <- mean(pred_cv == test_cl$categoria_precio)
accuracy_cv
```

```
## [1] 0.5729555
```

```
cat("Accuracy en prueba:", round(accuracy_cv * 100, 2), "%\n\n")
```

```
## Accuracy en prueba: 57.3 %
```

10.2 ¿Le fue mejor que al modelo anterior?

El modelo entrenado con validación cruzada obtuvo una precisión de aproximadamente 57.3%, la cual es ligeramente inferior al modelo anterior sin validación cruzada (61.3%).

Esta diferencia sugiere que el modelo inicial podría estar presentando cierto grado de sobreajuste, logrando un mejor desempeño en el conjunto de prueba pero menor capacidad de generalización.

La validación cruzada permite entrenar el modelo en múltiples subconjuntos de los datos, lo que genera un modelo más robusto y menos dependiente de una partición específica.

Aunque la precisión disminuye ligeramente, el modelo con validación cruzada es más confiable para su uso en escenarios reales, ya que ofrece una mejor estimación del desempeño en datos no vistos.

11 Haga al menos, 3 modelos más, cambiando la profundidad del árbol. ¿Cuál funcionó mejor?

```
# Modelos con distintas profundidades
arbol_d3 <- rpart(
  categoria_precio ~ .,
  data = train_cl,
  method = "class",
  control = rpart.control(maxdepth = 3, cp = 0)
)

arbol_d5 <- rpart(
  categoria_precio ~ .,
  data = train_cl,
```

```r
  method = "class",
  control = rpart.control(maxdepth = 5, cp = 0)
)

arbol_d8 <- rpart(
  categoria_precio ~ .,
  data = train_cl,
  method = "class",
  control = rpart.control(maxdepth = 8, cp = 0)
)

# Predicciones
pred_d3 <- predict(arbol_d3, test_cl, type = "class")
pred_d5 <- predict(arbol_d5, test_cl, type = "class")
pred_d8 <- predict(arbol_d8, test_cl, type = "class")

# Accuracy
acc_d3 <- mean(pred_d3 == test_cl$categoria_precio)
acc_d5 <- mean(pred_d5 == test_cl$categoria_precio)
acc_d8 <- mean(pred_d8 == test_cl$categoria_precio)

cat("Accuracy profundidad 3:", round(acc_d3*100,2), "%\n")
```

```
## Accuracy profundidad 3: 58.56 %
```

```r
cat("Accuracy profundidad 5:", round(acc_d5*100,2), "%\n")
```

```
## Accuracy profundidad 5: 59.82 %
```

```r
cat("Accuracy profundidad 8:", round(acc_d8*100,2), "%\n")
```

```
## Accuracy profundidad 8: 61.36 %
```

Se observa que al incrementar la profundidad del árbol, el accuracy mejora progresivamente. El modelo con profundidad 3 presenta el menor desempeño, lo que indica un caso de subajuste, ya que el modelo es demasiado simple para capturar los patrones de los datos.

Al aumentar la profundidad a 5, el modelo mejora su capacidad predictiva, logrando un mejor equilibrio entre simplicidad y ajuste.

El mejor resultado se obtiene con profundidad 8 (61.36%), lo que indica que un árbol más complejo logra capturar mejor las relaciones entre las variables.

Sin embargo, el incremento en desempeño no es muy grande, lo que sugiere que aumentar aún más la profundidad podría no generar mejoras significativas y podría llevar a sobreajuste.

12 Repita los análisis usando random forest como algoritmo de predicción, explique sus resultados comparando ambos algoritmos.

```r
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.4.3

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
```

```
## Adjuntando el paquete: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
# Modelo 1
rf_100 <- randomForest(
  categoria_precio ~ .,
  data = train_cl,
  ntree = 100,
  mtry = 4
)

# Modelo 2
rf_200 <- randomForest(
  categoria_precio ~ .,
  data = train_cl,
  ntree = 200,
  mtry = 4
)

# Modelo 3
rf_m2 <- randomForest(
  categoria_precio ~ .,
  data = train_cl,
  ntree = 100,
  mtry = 2
)

# Predicciones
pred_rf1 <- predict(rf_100, test_cl)
pred_rf2 <- predict(rf_200, test_cl)
pred_rf3 <- predict(rf_m2, test_cl)

# Accuracy
acc_rf1 <- mean(pred_rf1 == test_cl$categoria_precio)
acc_rf2 <- mean(pred_rf2 == test_cl$categoria_precio)
acc_rf3 <- mean(pred_rf3 == test_cl$categoria_precio)

cat("RF (100 árboles):", round(acc_rf1*100,2), "%\n")
```

```
## RF (100 árboles): 66.73 %
```

```r
cat("RF (200 árboles):", round(acc_rf2*100,2), "%\n")
```

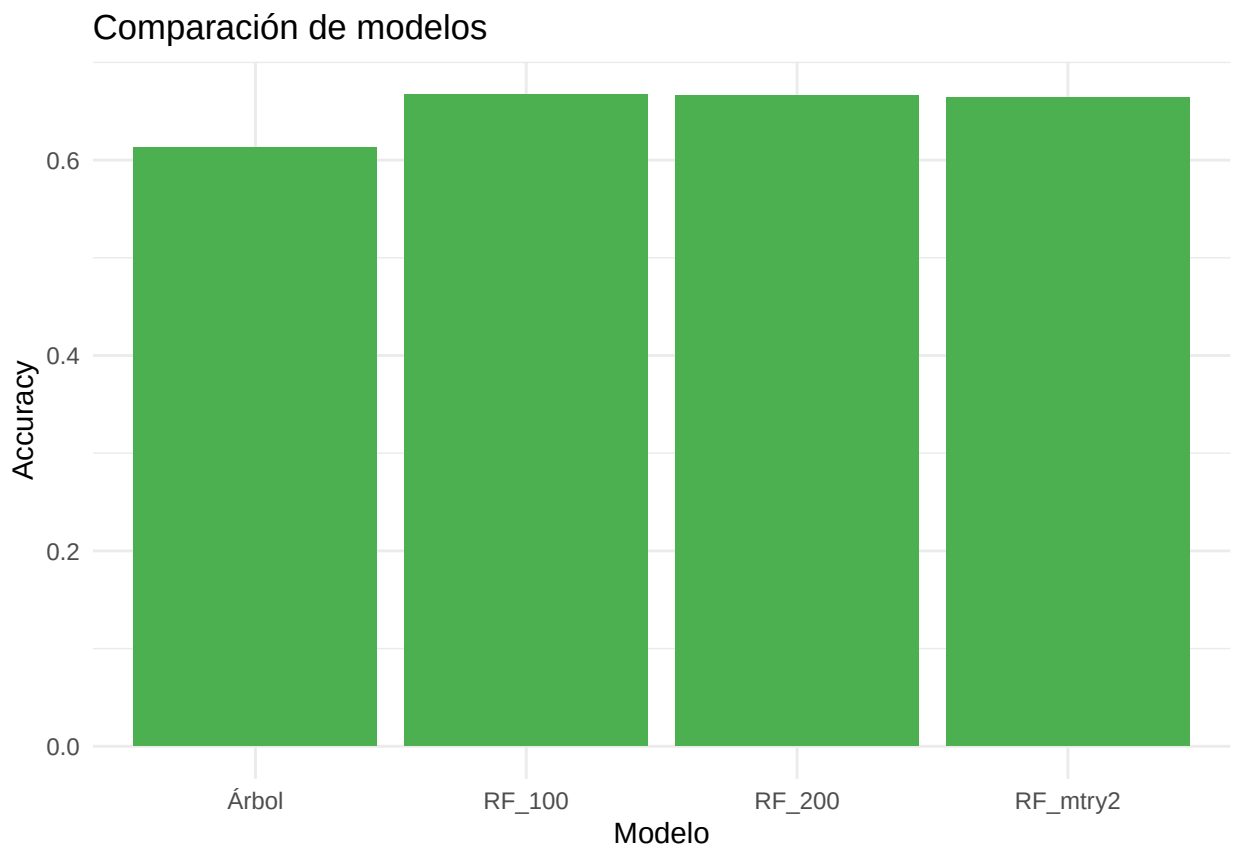
```
## RF (200 árboles): 66.63 %
```

```r
cat("RF (mtry=2):", round(acc_rf3*100,2), "%\n")
```

```
## RF (mtry=2): 66.42 %
```

```r
comparacion <- data.frame(
  Modelo = c("Árbol", "RF_100", "RF_200", "RF_mtry2"),
```

```r
  Accuracy = c(accuracy_cl, acc_rf1, acc_rf2, acc_rf3)
)

ggplot(comparacion, aes(x = Modelo, y = Accuracy)) +
  geom_col(fill = "#4CAF50") +
  labs(title = "Comparación de modelos",
       y = "Accuracy") +
  theme_minimal()
```



El modelo Random Forest muestra un desempeño superior al árbol de decisión individual, alcanzando un accuracy cercano al 66–67%, lo que representa una mejora aproximada de 5 puntos porcentuales.

Al comparar los distintos modelos:

Aumentar el número de árboles de 100 a 200 genera una leve mejora, lo que indica mayor estabilidad del modelo. Cambiar el parámetro mtry a 2 no mejora el desempeño, lo que sugiere que usar más variables en cada división permite capturar mejor la información.

En general, las diferencias entre configuraciones son pequeñas, lo que confirma que Random Forest es un modelo robusto y estable